

CHAPTER 5

RELATION WITH A REAL-WORLD PROBLEM

The manipulator studied in this thesis is not a laboratory abstraction. The UR5e is a commercially deployed collaborative arm, and machines in its class already do pick-and-place, machine tending, small-parts assembly, and camera-inspection work on production floors, as Chapter 1 describes. Safety on this class of arm is not a concern raised for the purposes of this thesis; it is an active requirement wherever the arm shares a workspace with a person rather than working behind a fence. The specific hazard addressed here, a manipulator configuration drifting toward a kinematic singularity or a joint limit during a reach-and-grasp motion, is exactly the kind of behaviour that is invisible to a person watching the arm move and only becomes apparent as a loss of control authority or an unexpected fault.

The engineering contribution of this thesis is not a new algorithm. Both PPO and PPO-Lagrangian are established methods, reviewed and specified in Chapters 2 and 3. What this thesis contributes is a way of stating the safety requirement that an integrator, not only a researcher, can work with directly. A reward-shaped safety term is a weight chosen by trial and error and folded into a single scalar that also carries the task reward, so nobody outside the training run can read off from it what the arm was actually asked to respect. A constraint is a number, the episodic cost budget d . It can be set from a measurement of the hardware and checked afterward against what the deployed policy actually did. Chapter 4 shows what that check looks like: at $d = 25$ the trained policy touched a joint limit in 0.00 % of 15,000 evaluation episodes, against 10.70 % for the unconstrained baseline, and reduced true singularity crossings by a factor of 7.0; tightening the same budget to $d = 15$ reduced singularity crossings by a factor of 39.9 and left joint-limit contact at zero. None of these are properties a reward weight can be audited against after the fact, because a reward weight was never a statement of a limit to begin with.

The result most relevant to an integrator, though, is not the headline safety numbers but the one in Section 4.6. Training the unconstrained baseline five times, changing nothing but the random seed, produced episodic safety costs ranging from 7.95 on one seed to 162.30 on another, a spread of more than twentyfold, with a matching lottery in joint-limit contact: one seed touched a joint limit on 47.83 % of its episodes, three never touched one at all. An integrator does not train five policies and average across them. They train one, and that one is whichever draw the random seed produced. The constrained agent pulled every seed into a band roughly two to five times as wide rather than twenty, at no measured cost to task performance. A stated budget therefore buys something beyond a lower safety cost on

average: the ability to say in advance what the shipped policy is likely to do, rather than finding out only after it has already been trained and deployed.

None of this should be read as a claim about a deployed system. Nothing in this thesis ran on the physical UR5e; the sim-to-real step is future work, discussed in Chapter 6. This chapter argues what the method makes possible, not what has been demonstrated on hardware, and that distinction has to hold for the strongest safety claim here as much as any other. Section 4.5 reports a counter-result alongside the reductions above: the worst single-episode manipulability measured on any arm, constrained or not, is identical. The constraint reduces how often the policy nears a singularity and how costly the typical episode is, but it does not make the rare worst-case excursion any shallower, at either budget. A hardware safety case built on this method would have to rest on expected exposure, the frequency and typical severity of near-singular operation, rather than on a claim that the worst case has been bounded, and it would still need whatever instantaneous protection a deployed system requires [?].

That predictability carries a socio-economic argument beyond the engineering one. A collaborative arm is attractive to a smaller manufacturer because it can run without a fenced cell or a dedicated safety engineer, which is also why UR-class arms are common outside large firms with in-house robotics expertise. A safety requirement expressed as an auditable number fits that market: a specification an integrator can check against a policy’s evaluation record is one a smaller operation can still verify, rather than one that requires reverse-engineering what a reward function was actually trained to trade off. Reducing how often an arm approaches a joint limit or a near-singular configuration also reduces a source of unplanned downtime and mechanical wear. None of this rests on the specific numbers measured here; it follows from stating the requirement as a constraint at all.

Mapped against the United Nations Sustainable Development Goals, this work sits most defensibly under two. SDG 9, Industry, Innovation and Infrastructure, is the direct fit: this is a method for making automation on already-deployed industrial hardware more auditable and more predictable, which is the kind of upgrade Target 9.4 asks industry to make. SDG 8, Decent Work and Economic Growth, applies through Target 8.8, which calls for safe working environments for all workers: the entire premise of a collaborative arm is that it shares space with a person, and a policy whose safety behaviour is stated as a checkable budget rather than hidden inside a reward weight is a step toward being able to certify that shared space as safe. Other goals in this family, such as SDG 12 on responsible production, would need a claim about resource use or waste that this thesis does not make, and are left out rather than listed for completeness.